

그래프 응용

그래프 응용

- 가중치?

- 간선에 값이 부여된 그래프

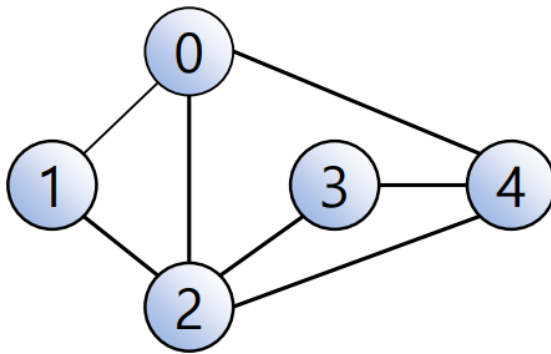
- 신장트리?

- 한 그래프에서 모든 정점을 포함하는 그래프
- 모든 정점들이 연결되어 있어야 하고, 사이클을 포함하지 않아야 한다

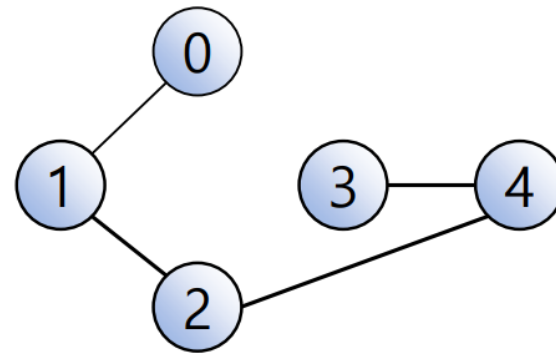
- 최소비용 신장트리?

- 간선들의 합이 최소가 되게 구성하는 그래프

그래프 응용

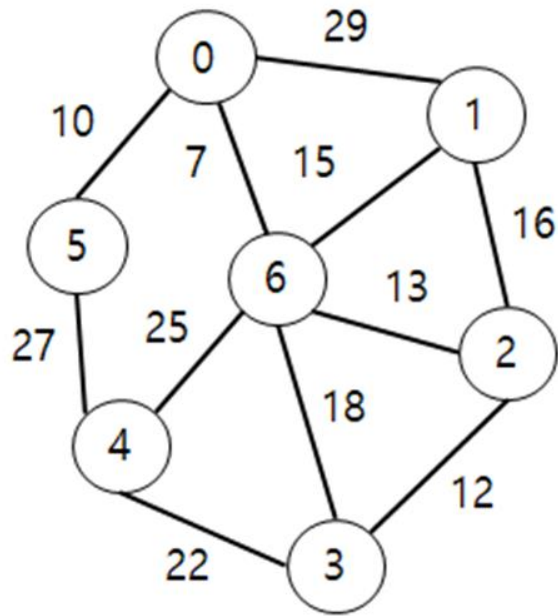


연결그래프



신장트리 1

그래프 응용



(그림 10.1) 가중치 그래프의 예

	0	1	2	3	4	5	6
0	0	29	∞	∞	∞	10	7
1	29	0	16	∞	∞	∞	15
2	∞	16	0	12	∞	∞	13
3	∞	∞	12	0	22	∞	18
4	∞	∞	∞	22	0	27	25
5	10	∞	∞	∞	27	0	∞
6	7	15	13	18	25	∞	0

(그림 10.2) (그림 10.1)의 인접행렬 표현

최소비용 신장트리

- 조건

- 그래프에 존재하는 모든 정점을 포함하여야 한다.
- 신장트리에 포함되는 간선의 가중치 합이 최소가 되어야 한다.
- 신장트리에서 두 정점 간의 경로가 존재하여야 한다.

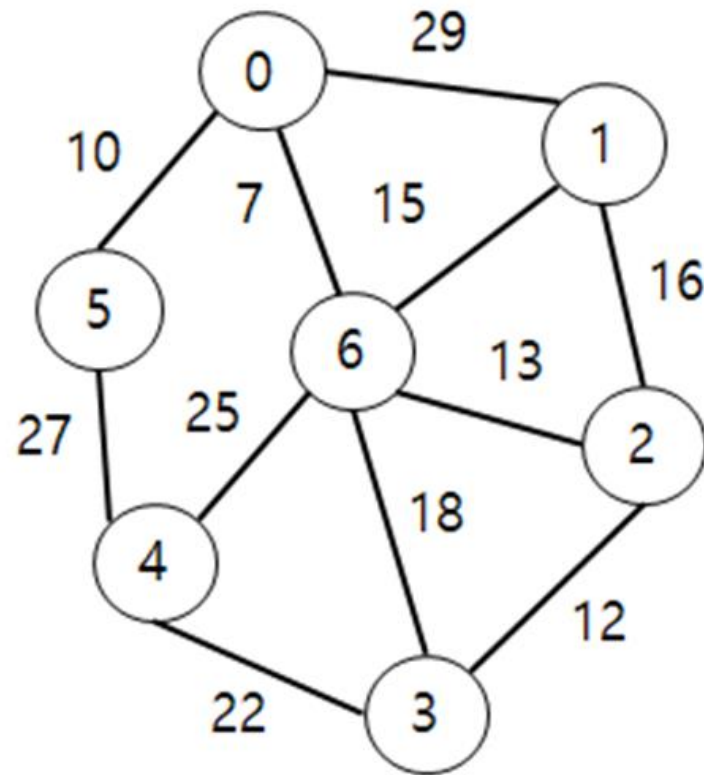
최소비용 그래프

- 표현 방법
- - Prim
- - Kruskal

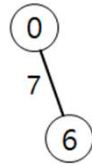
Prim

- 시작 정점을 선택한다. 시작 정점에 인접한 간선들 중에서 가장 작은 가중치 값을 갖는 간선을 선택한다.
- 시작 정점과 선택된 정점을 하나의 집합으로 하여 이 집합에 인접한 간선들 중에서 다시 가장 작은 가중치 값을 갖는 간선을 선택한다.
- 이러한 과정을 모든 정점이 포함될 때까지 반복한다. 즉, 그래프의 정점의 개수가 n 이라 하면 최종적으로 n 개의 정점을 포함하고 이 정점들을 $n-1$ 개의 간선으로 연결하는 최소비용 신장트리를 구성한다.

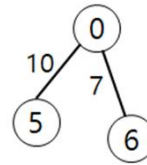
Prim



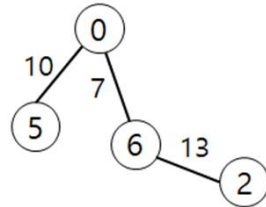
Prim



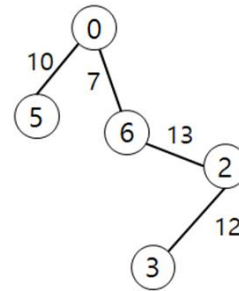
(1)



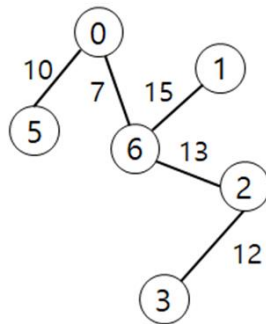
(2)



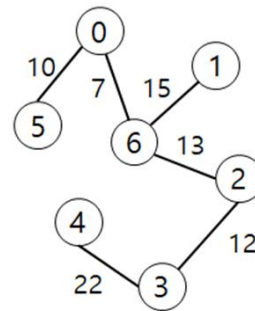
(3)



(4)



(5)



(6)

Prim

1. Algorithm MST_prim(weight, n)
2. Input: 주어진 그래프의 인접행렬(weight), 정점의 개수(n)
3. Output: 최소비용 신장트리
4. create selected[n] // 최소 비용으로 선택된 정점을 저장하는 배열
5. for i <- 0 to n-1 do
6. selected[i] <- 0
7. edge_count <- 0
8. selected[0] <- 1 // 시작 정점을 0으로 설정

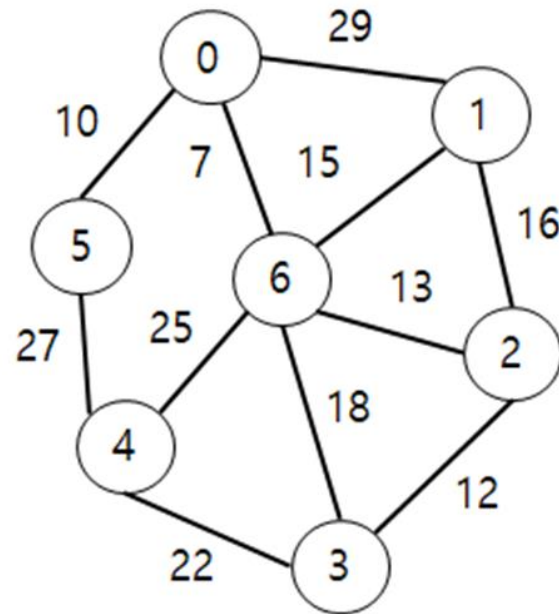
Prim

```
9. while (edge_count < n-1) do {
10.   min <- ∞ // 가장 큰 정수 값을 설정
11.   u <- 0
12.   v <- 0
13.   for i <- 0 to n-1 do {
14.     if (selected[i] = 1) {
15.       for j <- 0 to n-1 do {
16.         if ((selected[j] = 0) and weight[i, j]) {
17.           if (min > weight[i, j]) {
18.             min <- weight[i, j]
19.             u <- i
20.             v <- j }
21.           }
22.         }
23.       }
24.     }
25.     selected[v] <- 1
26.     print u, v, weight[u, v]
27.     edge_count <- edge_count + 1
28.   }
29. end
```

Kruskal

- 무게가 작은 간선들부터 MST에 포함시키는 greedy method 적용
- 그래프의 간선들을 무게의 오름차순으로 정렬
- 순서대로 간선을 꺼내어 현재의 MST에 추가했을 때 사이클이 형성되는지 검사
- 사이클을 형성하면 제외시키고, 그렇지 않으면 MST에 추가

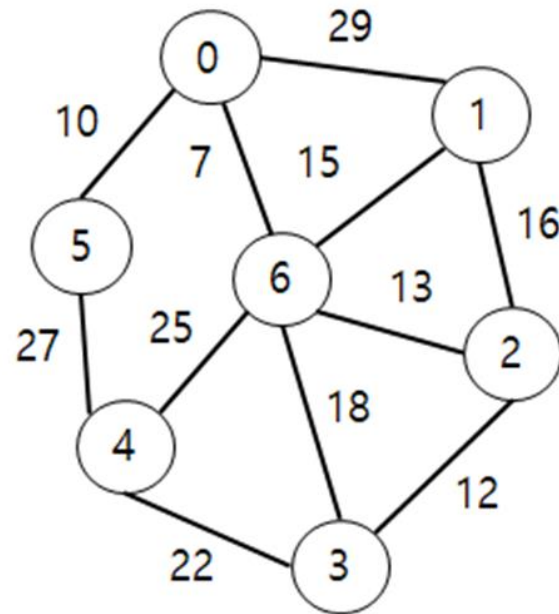
Kruskal



Kruskal

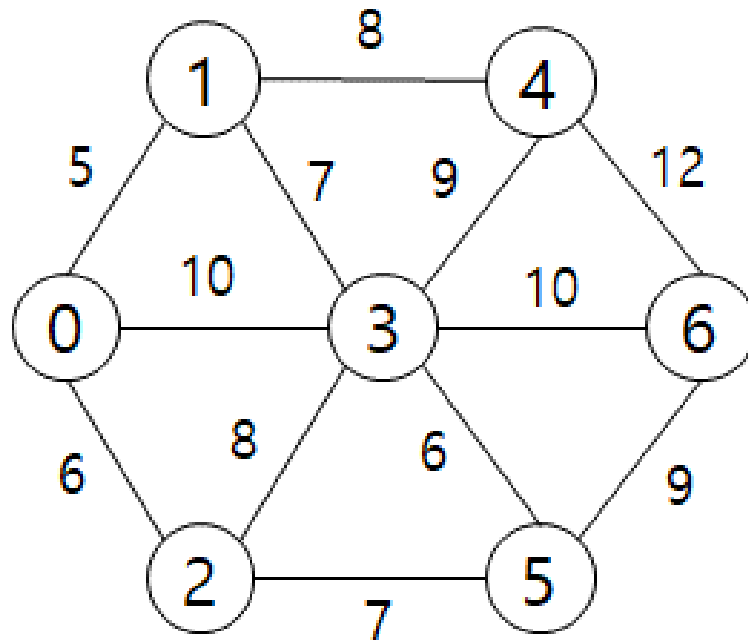
```
kruskal(G)
  // G = (V, E)
  E의 간선  $e_i$ 들을  $w(e_{i_1}) \leq \dots \leq w(e_{i_{|E|}})$  가 되도록 정렬;
   $E_T \leftarrow \phi$ ;
  ecount  $\leftarrow 0$ ;
  k  $\leftarrow 0$ ;
  while (ecount <  $|V| - 1$  ) do
    k  $\leftarrow k+1$ ;
    if (  $e_{i_k}$ 를  $E_T$ 에 넣었을 때 사이클이 발생하지 않음) then
       $E_T \leftarrow E_T \cup \{e_{i_k}\}$ ;
      ecount = ecount + 1;
  return  $E_T$ 
```

Union-find

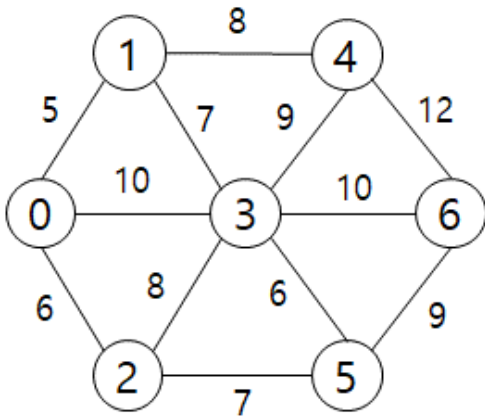


최단 경로를 찾기

- Dijkstra 알고리즘

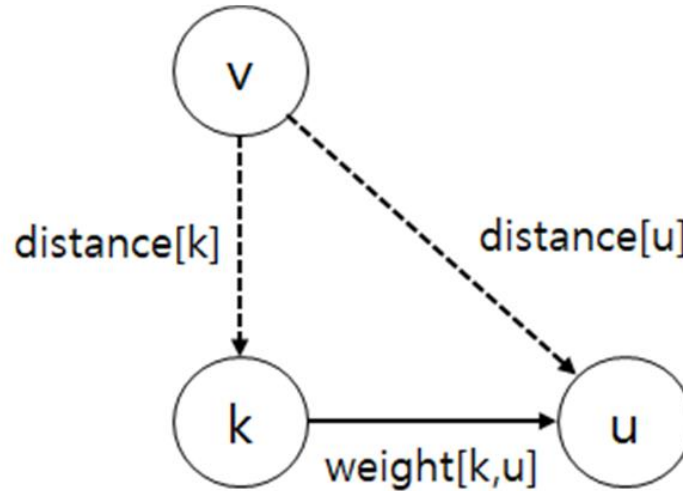


Dijkstra



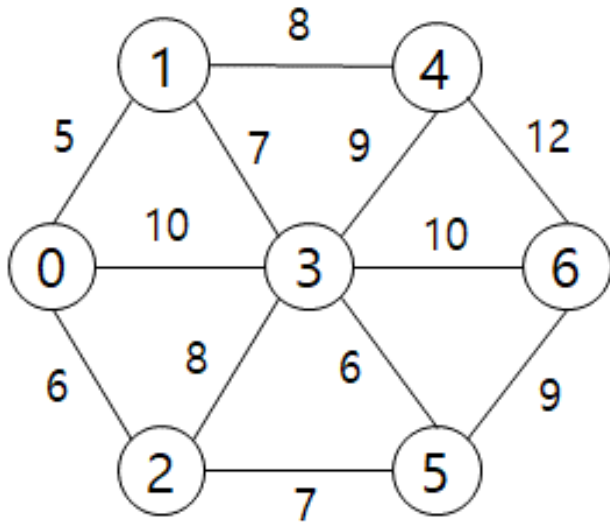
	0	1	2	3	4	5	6
0	0	5	6	10	∞	∞	∞
1	5	0	∞	7	8	∞	∞
2	6	∞	0	8	∞	7	∞
3	10	7	8	0	9	6	10
4	∞	8	∞	9	0	∞	12
5	∞	∞	7	6	∞	0	9
6	∞	∞	∞	10	12	9	0

Dijkstra



(그림 10.6) 시작 정점 v에서 정점 u까지 비용

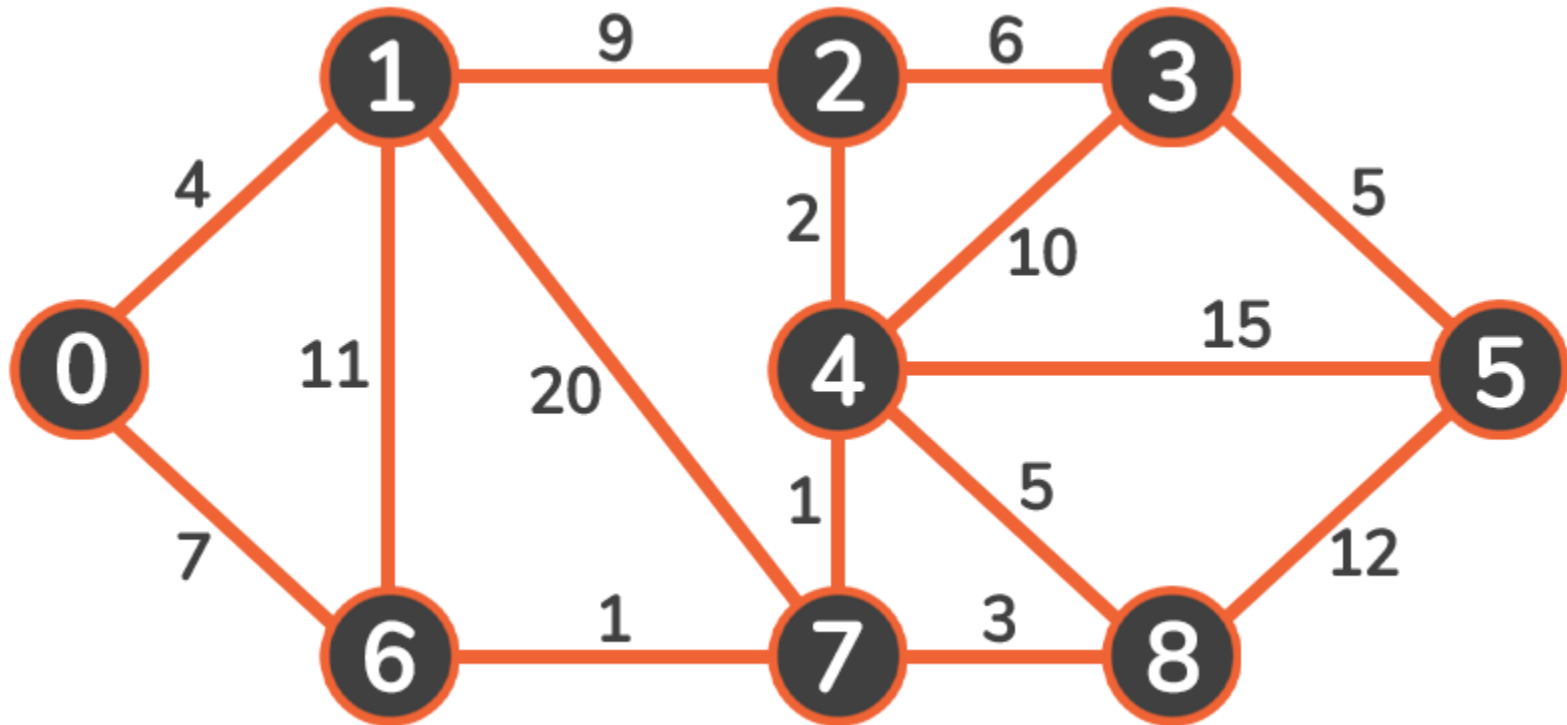
$$\text{distance}[u] = \min(\text{distance}[u], \text{distance}[k] + \text{weight}[k, u])$$



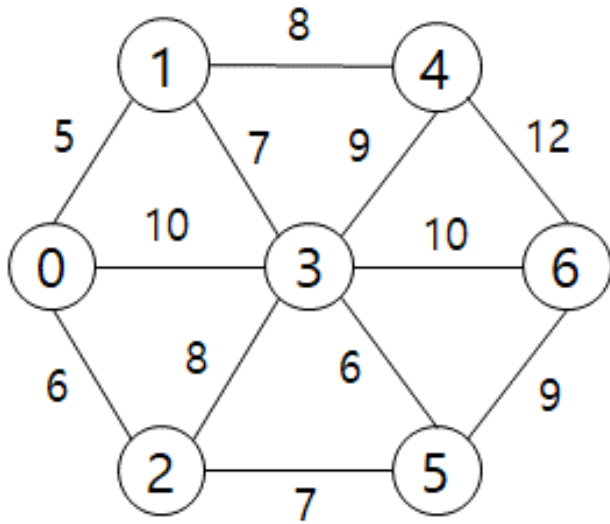
Dijkstra

distance[]	0	1	2	3	4	5	6	집합 S
(1)								
(2)								
(3)								
(4)								
(5)								
(6)								
(7)								

Dijkstra



출처 : ezerror



Dijkstra

최소비용 경로를 기억하기 위한 과정

	0	1	2	3	4	5	6	집합 S
(1)	<u>0</u>	5	6	10	∞	∞	∞	{0}
	0	0	0	0	0	0	0	
(2)	<u>0</u>	<u>5</u>	6	10	13	∞	∞	{0, 1}
	0	0	0	0	1	0	0	
(3)	<u>0</u>	<u>5</u>	<u>6</u>	10	13	13	∞	{0, 1, 2}
	0	0	0	0	1	2	0	
(4)	<u>0</u>	<u>5</u>	<u>6</u>	<u>10</u>	13	13	20	{0, 1, 2, 3}
	0	0	0	0	1	2	3	
(5)	<u>0</u>	<u>5</u>	<u>6</u>	<u>10</u>	13	<u>13</u>	20	{0, 1, 2, 3, 5}
	0	0	0	0	1	2	3	
(6)	<u>0</u>	<u>5</u>	<u>6</u>	<u>10</u>	<u>13</u>	<u>13</u>	20	{0, 1, 2, 3, 5, 4}
	0	0	0	0	1	2	3	
(7)	<u>0</u>	<u>5</u>	<u>6</u>	<u>10</u>	<u>13</u>	<u>13</u>	<u>20</u>	{0, 1, 2, 3, 5, 4, 6}
	0	0	0	0	1	2	3	

Dijkstra

1. Algorithm Shortest_path(weight, n, v)
2. Input: 그래프의 인접행렬(weight), 정점의 개수(n), 시작정점(v)
3. Output: 시작정점 v에서 모든 정점까지 최소비용과 경로
4. create distance[n], path[n], selected[n]
5. for i <- 0 to n-1 do {
6. distance[i] <- weight[v,i]
7. path[i] <- v
8. selected[i] <- 0 }
9. selected[v] <- 1
10. distance[v] <- 0
11. step <- 1

Dijkstra

```
12. while (step < n) do {
13.   min <- ∞ // 가장 큰 정수 값으로 설정
14.   for i <- 0 to n-1 do
15.     if ((distance[i] < min) and (selected[i] = 0)) {
16.       min <- distance[i]
17.       u <- i }
18.   selected[u] <- 1
19.   for w <- 0 to n-1 do
20.     if (selected[w] = 0)
21.       if ((distance[u]+weight[u,w]) < distance[w]) {
22.         distance[w] <- distance[u]+weight[u,w]
23.         path[w] <- u }
24.   step <- step + 1
25. }
26. for i <- 0 to n-1 do {
27.   print distance[i]
28.   print path[i] }
29. end
```

Dijkstra

```
12. while (step < n) do {
13.   min <- ∞ // 가장 큰 정수 값으로 설정
14.   for i <- 0 to n-1 do
15.     if ((distance[i] < min) and (selected[i] = 0)) {
16.       min <- distance[i]
17.       u <- i }
18.   selected[u] <- 1
19.   for w <- 0 to n-1 do
20.     if (selected[w] = 0)
21.       if ((distance[u]+weight[u,w]) < distance[w]) {
22.         distance[w] <- distance[u]+weight[u,w]
23.         path[w] <- u }
24.   step <- step + 1
25. }
26. for i <- 0 to n-1 do {
27.   print distance[i]
28.   print path[i] }
29. end
```